Image Preprocessing and Module Interface Design for Handwritten Text and Mathematical Symbol Recognition

Project Update Report — Individual Contribution

Mohammad Imtiaz Hassan (ID: 2321196642)
Department of Electrical and Computer Engineering, North South University, Dhaka, Bangladesh

***Abstract*—This report describes my contribution to a CSE445 group project that recognizes handwritten English characters and mathematical symbols through a web application. My responsibility is the image preprocessing module and the integration of the recognition modules. So far I have (i) defined the data interface shared by all modules, (ii) implemented and unit-tested the first two preprocessing stages—grayscale loading with input validation and threshold-based ink segmentation—and (iii) set up the repository structure, environment and automated testing workflow. All 14 automated tests pass. No classification results are reported yet, as the classifiers are scheduled for later weeks.**

***Index Terms*—handwriting recognition, image preprocessing, thresholding, software interface, unit testing.**

I. INTRODUCTION

Handwritten input from a user differs strongly from the clean images that recognition models are trained on: photographs contain uneven backgrounds, different pen colours and stroke widths, and symbols that are not centred. A classifier performs reliably only when its input follows the distribution of its training data, so every image must be converted into one standard format before prediction.

The project has four modules: a text recognizer trained on EMNIST [1], a mathematical symbol recognizer trained on CROHME [2], a Streamlit user interface, and the preprocessing and integration module described here. A user uploads an image and selects text or math mode; my module converts the image and routes it to the matching recognizer:

```
upload → run_pipeline(image, mode) →
preprocess_image(image, size) → recognize_text /
recognize_math → label + confidence
```

Section II presents the interface, Section III the implementation, Section IV its verification, Section V the repository work, and Section VI the limitations and next steps.

II. INTERFACE DESIGN

Because four members develop modules in parallel, the format of the data passed between them must be fixed in advance. I defined the contract in Table I and documented it in the repository (docs/architecture.md). The raw image enters only run_pipeline; the classifiers receive only the processed array, never a raw photograph.

TABLE I
AGREED INPUT FORMAT FOR ALL RECOGNIZERS

Property	Value	Reason
Type	NumPy array	Direct input to PyTorch and scikit-learn
Shape	(size, size)	One grayscale channel; colour carries no class information
Dtype	float32	Default floating-point type in PyTorch
Range	[0, 1]	Small inputs give more stable gradient-based training
Colours	ink 1.0, background 0.0	Matches EMNIST (white ink on black) [1]
Size	per recognizer	EMNIST uses 28×28; each model declares INPUT_SIZE

Pixel intensities $x \in \{0, \ldots, 255\}$ are mapped to the agreed range by

$$x' = x / 255 \quad (1)$$

This scaling is applied only in the final preprocessing stage, so that one stage alone controls the output format and intermediate stages can use Pillow operations on 8-bit images.

III. IMPLEMENTATION

A. Grayscale Loading and Input Validation

The function `load_grayscale` accepts a file path or a PIL image, converts it with Pillow's "L" mode and returns a 2-D `uint8` NumPy array. Pillow computes the luminance of each pixel with the ITU-R 601-2 transform [5]:

$$Y = 0.299R + 0.587G + 0.114B \quad (2)$$

Grayscale removes colour, which is irrelevant to the identity of a symbol, and reduces the input from three values per pixel to one. The returned array has shape (height, width), whereas Pillow reports sizes as (width, height); this difference is documented because it is a common source of errors. Invalid input raises a TypeError (wrong type) or a ValueError (image with zero height or width), so that integration errors are reported with a clear message instead of failing inside later stages.

B. Ink Segmentation by Thresholding

The function `to_ink_mask` separates ink from background with a global threshold T (default 128):

$$M(i, j) = 1 \text{ if } Y(i, j) < T, \text{ otherwise } 0 \quad (3)$$

The comparison is applied to the whole array at once using NumPy vectorised operations [4], giving a Boolean mask of the same shape. A useful property of this mask is that ink is 1 and background is 0, which already matches the agreed white-ink-on-black convention of Table I; dark-on-light photographs therefore need no separate inversion step. Fig. 1 shows both stages on a synthetic test image.

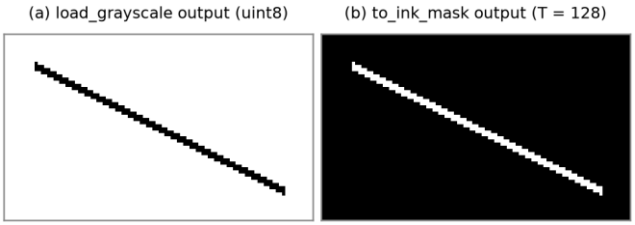


Fig. 1. Output of the implemented stages on a 100×60 test image with a black line: (a) grayscale array, values 0–255; (b) ink mask, 203 ink pixels.

IV. VERIFICATION

Each stage is verified with automated unit tests written with pytest [6]. Six tests cover the preprocessing module: output shape (60, 100), dtype uint8 and value range 0–255 of load_grayscale; a TypeError for a non-image input; a ValueError for an empty image; the mask dtype and its values at a background pixel and a line pixel; and the effect of the threshold parameter on a uniform grey image (intensity 100 is ink at T = 128 and background at T = 50). All 14 tests in the repository pass locally and are executed automatically by GitHub Actions on every push and pull request.

Edge-case testing found a real defect during development: an early return statement made the empty-image check unreachable, so empty images passed silently. The defect was invisible with normal inputs and was only revealed by the dedicated empty-image test, after which it was fixed.

V. REPOSITORY AND WORKFLOW

I restructured the shared repository to the course-mandated layout (main.py, support/, data/, others/), fixed the resulting test and import failures, and merged the changes through reviewed pull requests (#3–#5) with passing automated checks. I prepared the 8-week project plan (plan.md), the dependency list (requirements.txt) and the interface document, drafted with AI assistance [8]. Python 3.12 was selected for all members after installation on Python 3.15 failed because no PyTorch build was available for that version. Each member works on a personal branch, and code reaches the main branch only through pull requests.

VI. LIMITATIONS AND NEXT STEPS

A fixed global threshold is unreliable for photographs with shadows or uneven lighting, where parts of the background fall below T. I plan to compare it with Otsu's method [3], which selects T per image from its intensity histogram. The remaining stages of preprocess_image are: cropping to the bounding box of the ink, padding to a square to preserve the aspect ratio, resizing to the recognizer's input size and scaling to [0, 1]. Afterwards, run_pipeline will route images to the two recognizers, and the module will be tested on real handwriting photographs. Following the project plan, preprocessing is due by Week 2 and integration by Week 5. So far, all results are from synthetic test images; no recognition accuracy is claimed.

REFERENCES

[1] G. Cohen, S. Afshar, J. Tapson, and A. van Schaik, “EMNIST: Extending MNIST to handwritten letters,” in Proc. Int. Joint Conf. Neural Networks (IJCNN), 2017, pp. 2921–2926.
[2] M. Mahdavi, R. Zanibbi, H. Mouchère, C. Viard-Gaudin, and U. Garain, “ICDAR 2019 CROHME + TFD: Competition on recognition of handwritten mathematical expressions and typeset formula detection,” in Proc. ICDAR, 2019, pp. 1533–1538.
[3] N. Otsu, “A threshold selection method from gray-level histograms,” IEEE Trans. Syst., Man, Cybern., vol. SMC-9, no. 1, pp. 62–66, 1979.
[4] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020.
[5] Pillow (PIL Fork) documentation, “Image.convert.” [Online]. Available: https://pillow.readthedocs.io
[6] pytest documentation. [Online]. Available: https://docs.pytest.org
[7] OpenAI, ChatGPT, used for the initial team project plan, Oct. 2026.
[8] Anthropic, Claude, AI assistant used as a tutor for concept explanations, code review, documentation drafting and report preparation, Oct. 2026.